Test Plan Document

For

Indiana University Southeast

1 **INTRODUCTION**..... ERROR! NO BOOKMARK NAME GIVEN.
2 **BUSINESS BACKGROUND**..... ERROR! NO BOOKMARK NAME GIVEN.
3 **TEST OBJECTIVES**..... ERROR! NO BOOKMARK NAME GIVEN.
4 **SCOPE**..... ERROR! NO BOOKMARK NAME GIVEN.
5 **TEST TYPES IDENTIFIED**..... ERROR! NO BOOKMARK NAME GIVEN.
6 **PROBLEMS PERCEIVED**..... ERROR! NO BOOKMARK NAME GIVEN.
7 **ARCHITECTURE**..... ERROR! NO BOOKMARK NAME GIVEN.
8 **ENVIRONMENT**..... ERROR! NO BOOKMARK NAME GIVEN.
9 **ASSUMPTIONS**..... ERROR! NO BOOKMARK NAME GIVEN.
10 **FUNCTIONALITY**..... ERROR! NO BOOKMARK NAME GIVEN.
11 **SECURITY**..... ERROR! NO BOOKMARK NAME GIVEN.
12 **PERFORMANCE**..... ERROR! NO BOOKMARK NAME GIVEN.
13 **USABILITY**..... ERROR! NO BOOKMARK NAME GIVEN.
14 **TEST TEAM ORGANIZATION**..... ERROR! NO BOOKMARK NAME GIVEN.
15 **SCHEDULE**..... ERROR! NO BOOKMARK NAME GIVEN.
16 **DEFECTS CLASSIFICATION MECHANISM**..... ERROR! NO BOOKMARK NAME GIVEN.
17 **CONFIGURATION MANAGEMENT**..... ERROR! NO BOOKMARK NAME GIVEN.
18 **RELEASE CRITERIA**..... ERROR! NO BOOKMARK NAME GIVEN.

Test Plan

1 Introduction

The purpose of this Test Plan is to define the strategy and objectives for testing the Lost Grenadiers campus navigation system. This software is designed to assist students, parents, and first-time visitors with navigating the Indiana University Southeast campus efficiently and confidently. The system provides an interactive map, detailed routing to buildings and rooms, and shortest-path calculations.

This document outlines the approach that will ensure the software meets its functional performance and security requirements. It will also serve as a guide for the team to execute tests systematically, document results, and correct flaws.

2 Business Background

Navigating a university campus can be challenging, especially for students with anxiety, visiting parents, and new students and faculty. Misunderstandings or inefficiencies in finding classrooms, dorms, or offices may lead to stress and decreased engagement with campus life.

The Lost Grenadiers campus navigation system addresses these issues by providing an interactive and user-friendly map with routing functionality. Using a combination of client-side mapping libraries and server-side shortest-path algorithms, the system allows users to quickly locate the destination while following an optimal path. While the primary userbase is students, the system is also suitable for staff, faculty, and the general public.

Currently, the system is deployed in a local Proxmox virtualized environment for development and testing purposes, but the infrastructure is designed to be portable for future deployment on campus servers or other environments. By providing reliable navigation support, the system aims to enhance the overall campus experience and reduce the stress associated with pathfinding.

3 Test Objectives

The objective of testing the Lost Grenadiers campus navigation system is to ensure that the software meets functionality, security, and usability requirements while operating within the allotted environment. Specifically, the test objectives are:

1. **Functionality Verification**
   - Ensure accurate routing from any campus location to a selected destination, including accessibility-aware paths
   - Verify proper display of buildings, rooms, and paths on the Leaflet map
   - Confirm SQL queries via pgRouting return correct shortest-path results
   - Validate that malformed API calls return gracefully
2. **Security Assurance**
   - Validate SSL/TLS encryption using Let’s Encrypt certificates
   - Confirm SQL queries are protected against SQL injection attacks

- Ensure API access is restricted to authorized clients

3. **Usability Assessment**
   - Verify that users can easily locate and select destinations through the browser interface
   - Ensure accessibility options are correct and intuitive
   - Confirm system feedback, such as errors and empty results, are clear and informative

4 Scope

Inclusions

The testing effort will cover the following components and functionalities:

- **Frontend (Leaflet.js Map)**: User interaction, route display, search bar functionality
- **API Gateway (Flask Server)**: Request validation, route computation, JSON responses, error handling
- **Routing Engine (pgRouting / PostGIS / PostgreSQL)**: Shortest-path calculations, accessibility filters, data integrity
- **Security Measures**: SSL/TLS encryption, injection prevention, firewall/DMZ compliance
- **Performance Metrics:** Response time for routing requests, database query execution

Exclusions

The following aspects are outside the scope of this test plan:

- External GPS signal quality or reception issues
- Third-party mapping correctness (OpenStreetMap tiles accuracy)
- Server hardware failures beyond Proxmox level faults
- User authentication, as no login system exists

5 Test types Identified

The following types of testing will be conducted to ensure that the Lost Grenadiers campus navigation system meets all functional and non-functional requirements:

1. **Functional Testing**
   - Verify all core features work as intended, including map rendering, building and room searches, routing calculations, and accessibility filtering
   - Test API endpoints to ensure correct responses and proper error handling for invalid inputs
2. **Performance Testing**
   - High node density may impact route computation and map rendering speed
   - Backup and graph maintenance procedures could temporarily affect database performance
3. **Security Testing**

- Validate SSL/TLS encryption for all communications between client and server
- Test for vulnerabilities such as SQL injection in routing queries and API request handling
- Confirm API access is properly restricted and that sensitive operations are secure (backups, graph maintenance)

4. **Compatibility Testing**
   - Verify the system functions across supported browsers and operating systems
   - Ensure the Flask backend, PostGIS database, and Leaflet frontend operate correctly in the Proxmox environment

6 Problems Perceived

The following potential issues and risks have been identified in the Lost Grenadiers campus navigation system:

1. **Data Integrity Risks**
   - Incorrect or incomplete graph data could cause routing errors
   - Pathfinding may fail if nodes are incorrectly connected
2. **Performance Constraints**
   - Large datasets and node density may impact route computation and rendering speed
   - Backup and graph maintenance could affect availability and uptime
3. **Security Vulnerabilities**
   - Potential SQL injection if query sanitization fails
   - Misconfiguration of SSL/TLS certificates or firewall/DMZ settings could compromise secure communication
4. **Deployment Limitations**
   - Current deployment is on a home Proxmox environment, which may replicate the campus server performance or network conditions
   - External factors like GPS signal quality are not controlled within this system
5. **Scalability Issues**
   - Leaflet map performance may degrade with increased concurrent users or with larger node datasets

Mitigation strategies will include input validation, performance benchmarking, incremental updates to the graph, and clear UI feedback.

7 Architecture

The Lost Grenadiers campus navigation system employs a client-server architecture designed for secure and scalable campus navigation. The system is composed of the following primary components:

1. **Frontend (UI/UX)**
   - **Technology**: HTML, CSS, JavaScript, Leaflet.js
   - **Purpose:** Provides an interactive map for users to search for rooms, buildings, or points of interest, and displays computed navigation routes

- **Interactions:** Sends user requests to the Flask API and renders GeoJSON route data returned from the server

2. **API Gateway (Flask)**
   - **Technology**: Python, Flask
   - **Purpose:** Serves as the intermediary between the frontend and backend database. Validates user inputs, handles API requests, and returns JSON-formatted route data
   - **Responsibilities**: Validating user coordinates and room identifiers, executing server-side routing queries via pgRouting, return results to frontend in a usable format
3. **Routing Engine**
   - **Technology**: PostgreSQL, PostGIS, pgRouting
   - **Purpose**: Calculates shortest paths between nodes using Djijstra's algorithm
   - **Data Structures**: TODO VALIDATE: Weighted graphs with nodes representing navigable pathways
   - **Responsibilities**: Perform shortest-path computations, support accessibility filters, maintain spatial data integrity
4. **Security and Authentication**
   - **Technology**: SSL/TLS via Let's Encrypt!
   - **Purpose**: Encrypts all communications between client and server
   - **Responsibilities**: Prevent unauthorized access, prevent SQLi via parameterized queries, restrict access behind firewall and DMZ isolation
5. **System Administration and DevOps**
   - **Technology**: Proxmox
   - **Purpose:** Hosts all containers and backends for bac

8 Environment

The testing environment for the Lost Grenadiers campus navigation system mirrors the intended deployment as closely as possible within current constraints.

1. **Hardware and Virtualization**
   - **Proxmox VE Host (Linux)**: Hosts containers for the Flask server and PostgreSQL database
   - **Container Resources:** CPU, memory, and storage allocation are limited to personal resources, but will flourish in production environments with more resources
2. **Software and Platforms**
   - **Backend**: Flask 2.x (Python 3.x), PostgreSQL 15, PostGIS, pgRouting
   - **Frontend**: Modern web browser (Chrome, Safari, Firefox) with Leaflet.js map interface
   - **Security**: Let's Encrypt SSL/TLS certificates, firewall and DMZ configuration
3. **Network**
   - Local LAN connecting Proxmox host to testing client machines
   - HTTPS required for all API communications
   - Network segmentation via Proxmox virtual network and DMZ layer to isolate backend services

9 Assumptions

These are conditions we assume to be true for testing purposes, based on the system design and expected environment.

1. **User Behavior Assumptions**
   - Users will input valid coordinates and destination identifiers within campus boundaries
   - Users may request accessible routes, and the system will honor these preferences
   - Users interact via a modern web browser with JavaScript enabled
2. **System Behavior Assumptions**
   - The Flask API endpoint is the single-entry point for all routing requests
   - SQL queries will always execute successfully if inputs are valid
   - GeoJSON responses are correctly formatted for Leaflet.js rendering
   - The database contains accurate and up-to-date data
3. **Environmental Assumptions**
   - HTTPS is required; any unencrypted HTTP requests are invalid
   - Network latency is within acceptable limits for the campus LAN
4. **Data and Maintenance Assumptions**
   - Graph maintenance and backup scripts are functional and will not interfere with live queries during normal testing
   - New building or node data is pre-validated (correct SRID, no dangling edges, proper accessibility flags)
5. **Security Assumptions**
   - All API requests are sent from authorized clients
   - Authentication is limited to API-level access
   - SQLi prevention via parameterized queries is implemented

10 Functionality

Constraints and Resolutions

Parameter	Customer Constraints	Infosys Limitations
Routing Accuracy	Must always return the shortest or optimal path between any two points on campus	Accuracy depends on the correctness of nodes and edges in the database.
Accessibility	Users must have the option to request accessible routes	Accessibility flags must be maintained
Map Rendering	Map must display all buildings, rooms, paths in browser	Leaflet.js performance may degrade with high node density. Older browsers may also struggle.
API Reliability	All routing requests must be served with valid JSON and quickly	System runs in a limited Proxmox environment, high concurrent requests may lower performance
Error Handling	Invalid requests must be handled gracefully	Backend may not handle unexpected or extreme edge cases unless tested

Risk Identified & Mitigation Planned

1. Incomplete graph data causes route errors
   - Topology validation and incremental updates, messages if route cannot be found
2. Accessibility flags missing on new nodes
   - Maintain a checklist when updating graph data
3. Leaflet map rendering slowly with large node count
   - Limit node count, delete older deprecated nodes
4. API failures with malformed input
   - Input validation, rate limiting, logging of failed requests
5. Unexpected database errors
   - Wrap all database calls in error handling routines, return user friends message

Test Strategy

1. **Manual Testing**
   - Verify map rendering, route computation, and accessibility filtering
   - Test routing from various start/destination points, including edge of campus and building interiors
   - Check error handling by sending invalid/malformed requests
2. **API Testing**
   - Send both valid and invalid requests to endpoint
   - Validate that JSON responses are correctly structured
   - Measure response times and check for errors
3. **Boundary and Edge Case Testing**
   - Start and end at the same location
   - Requests that require inaccessible paths when accessibility mode is enabled
   - Requests on unmapped or partially mapped nodes

Automation Plans

1. **API Automation**
   - Use Python scripts to automate repeated route requests
   - Validate JSON output, status codes, and error messages
   - Include automated tests for accessibility filter correctness
2. **Regression Testing**
   - Automatically verify that previous routes still return correct paths after database updates or after new nodes are added

Deliverables

1. Test case documentation for all routing and map features
2. Test summary report with pass/fail status for each functionality
3. Logs of successful/failed test cases

11 Security

Constraints and Resolutions

Parameter	Customer Constraints	Infosys Limitations
Encryption	All communications between client and server must be encrypted	Limited to SSL/TLS via Let's Encrypt: certificate renewal is necessary
SQLi Prevention	All routing queries must be protected from SQLi attacks	Only parameterized queries implemented, new SQL code must be checked
API Access Control	Only authorized clients can access sensitive endpoints	No login system, limited to firewall/DMZ restrictions
Backup Security	Database backups must be encrypted and rotated regularly	Backup scripts are manual
Server Hardening	Production-like security hardening should prevent unauthorized access	Testing done in Proxmox environment, not similar to campus production

Risk Identified & Mitigation Planned

1. SSL certificate misconfiguration could leak data
   - Verify certificates during testing, enforce HTTPS redirection
2. SQLi in new queries
   - Enforce code review with automated SQLi testing
3. Unauthorized API access
   - Test DMZ isolation, log all access attempts
4. Backup file compromise
   - Encrypt files, rotate regularly
5. Server misconfiguration
   - Maintain server hardening checklist, perform penetration testing on environment

Test Strategy

1. **Manual Security Testing**
   - Verify HTTPS communication
   - Attempt SQLi attacks on routing endpoints
   - Test DMZ isolation by attempting unauthorized API access
2. **Automated Security Testing**
   - Use scripts to send malicious inputs to API endpoints
   - Validate SSL/TLS config via SSL Labs

Automation Plans

1. Automated SQLi tests via Python or another security framework
2. Regular SSL validation scripts
3. Logging and alert scripts for unauthorized access

Deliverables

1. Security test case documentation
2. Defect reports for vulnerabilities discovered
3. Test summary reports with pass/fail status
4. Logs of malicious request tests, SSL verification results, firewall tests

12 Performance

Constraints and Resolutions

Parameter	Customer Constraints	Infosys Limitations
Routing Response Time	Route requests must respond quickly	Performance depends on node/edge density and container resources
Map Rendering	Map must remain responsive even with multiple layers/nodes	Leaflet.js may slow down when rendering many nodes
Database Query Execution	Queries must complete without timeout under standard load	PostgreSQL/pgRouting may be limited by container CPU/memory allocation
Concurrent Requests	System should handle multiple simultaneous requests	Testing environment may not simulate campus resources
Backup & Maintenance	Routine backups should not affect active queries	Backups run sequentially, may impact performance

Risk Identified & Mitigation Planned

1. High node density slows routing queries
   - Benchmark queries, precompute frequently used paths
2. Multiple simultaneous users may overload API
   - Simulate concurrency in testing, monitor response time and resources
3. Backup scripts affecting availability
   - Schedule backups during off-peak hours, test incremental backups
4. Unexpected Proxmox resource constraints
   - Monitor CPU/memory, scale resources if needed

Test Strategy

1. **Manual Performance Testing**
   - Measure routing response times for short and long paths
   - Test map responsiveness when displaying many nodes/edges
   - Execute multiple routing requests concurrently to observe latency
2. **Automated Performance Testing**
   - Benchmark queries and record execution times
   - Monitor server resource usage during peak simulated loads

Automation Plans

- Automated load tests using Python
- Scripts to repeatedly request routes and log response times

Deliverables

- Performance test case documentation
- Load and response time measurement logs
- Defect reports for slow queries, rendering lag, etc.

- Test summary report with performance benchmarks and recovering tips

13 Usability

Constraints and Resolutions

Parameter	Customer Constraints	Infosys Limitations
Ease of Navigation	User must be able to easily find buildings and routes without training	Interface complexity is limited by time and resources
Search Functionality	Users must be able to quickly locate rooms and buildings	Search depends on accurate naming conventions in database
Accessibility Options	Accessibility settings must be visible and easy to enable	Accessibility in UI is limited to a toggle
Clarity of Routes	Routes must be visually clear and easy to follow on the map	Dense node areas may cause route inconsistencies
Error Messaging	Users must receive clear and understandable feedback for invalid prompts	Error messages are text-based only
Responsiveness	Interface must remain responsive during map interaction and routing	Performance depends on browser and device processing power

Risk Identified & Mitigation Planned

1. Users become confused by interface layout
   - Perform UI testing, simplify where possible
2. Search results return unexpected or no matches
   - Test common search terms, add “no results found” message
3. Accessibility options are overlooked
   - Ensure toggle is clearly labelled and tested for visibility
4. Routes are hard to follow in dense areas
   - Adjust line thickness and zoom behavior, test routes at different zoom levels

Test Strategy

1. **Manual Usability Testing**
   - Observe users attempting common tasks
   - Verify first time users can navigate without instruction
   - Test error handling from a user perspective
2. **Cross-Browser Usability Testing**
   - Verify consistent UI across Chrome, Firefox, and Safari
   - Test different screen sizes and zoom levels

Automation Plans

- Automated UI tests to verify page loads and core controls appear
- Scripted checks for presence of UI elements
- Automated route rendering verification (route appears only after request)

Deliverables

- Usability test cases and test scripts
- Notes and observations from user walkthrough testing
- Defect reports for UI/UX issues and unclear behavior
- Test summary report documenting usability findings and improvements

14 Test Team Organization

The testing of the Lost Grenadiers campus navigation system is a collaborative effort involving clearly defined roles and responsibilities.

Role	**Name**	**Responsibilities**
Project Manager	Audrey Dickson	Oversee testing timeline, coordinates communication, approves results
Lead Developer	Kaden Langford	Maintains database and backend infrastructure
QA/Tester	Andrew Swartz	Design and execute test cases, perform functional, security, and performance testing, log creation
UI/UX	Audrey Dickson	Validates usability, accessibility, and UI behavior during testing
Security Architect	Andrew Swartz	Conducts penetration testing, verifies SSL/TLS, reviews firewall and API security

Schedule

The testing schedule aligns with the development milestones of the Lost Grenadiers project and follows an incremental testing approach to reduce risks and identify defects.

Phase	Activities	Estimated Duration
Test Planning	Finalize test plan, identify test cases, prepare environment	1 week
Environment Setup	Configure Proxmox containers, verify SSL/TLS, load test data	1 week
Functional Testing	Test routing, map rendering, accessibility, error handling	2 weeks
Security Testing	SQL injection testing, SSL verification, firewall/DMZ validation	1 week
Performance Test	Load testing, respons time benchmarking, rendering tests	1 week
Final Review and Release	Compile test reports, approve release	1 week

The table above is not a finalized version and may be altered depending on defect severity or infrastructure changes.

Defects Classification Mechanism

Type of Defects	Functionality	Performance	Security	Usability	Compatibility
Critical	Routing fails, system crash, wrong path returned	Extreme latency and timeouts	Data leakage, SQLi, SSL failure	UI unusable	System fails on supported browser or OS
Major	Accessibility feature fails	Noticeable slowdowns, still usable	Misconfigured firewall, incomplete access restriction	User confusion due to unclear layout	Broken feature on browsers
Minor	Small routing inaccuracies	Slight delays with no usability impact	Security warnings for end users	Minor UI inconsistencies	Layout issues on various screen sizes
Cosmetics	Visual inconsistencies in route	None	NA	Font, color, spacing, alignment, etc.	Visual differences between browsers

Defects Logging and Status Changing Mechanism

All defects discovered during testing will be logged in Github. The status of each defect can follow a lifecycle via the Projects tab, showing New, Assigned, In Progress, Fixed, Retest, Closed, and Reopened.

Turn Around Time for Defect Fixes

Defect resolution time is highly variable but are altered to ensure that critical issues are resolved promptly.

Severity	**Expected Fix Time**
Critical	24-48 hours
Major	3-5 days
Minor	1-2 weeks
Cosmetic	As time permits

Configuration Management

The configuration management process ensures that all components of the Lost Grenadiers campus navigation system are consistently maintained, versioned, and tracked throughout testing and development.

Key Activities:

- Version Control: All source code is maintained in Git with proper branching and commit messages
- Data Management: Database schema and table data are version controlled and backed up
- Change Control: Any updates to code, configuration, or graph data require review, approval, and documentation in Github.
- Documentation: Test plans, test cases, defect logs, etc., are stored in a centralized repo for audit and traceability

15 Release Criteria

The release of the Lost Grenadiers campus navigation system to production will be approved only when the following criteria are met:

1. **Functional Completeness**: All core features (routing, map rendering, accessibility filtering) pass predefined test cases, and no critical defects remain unresolved
2. **Security Validation**: SSL/TLS certificates are correctly verified, and all API endpoints are protected against SQLi and unauthorized access
3. **Performance Requirements:** Routing queries return results within acceptable response times and the Leaflet map is responsive.
4. **Configuration and Data Integrity**: Database schema and nodes/edges are up-to-date, and backup mechanisms are operational.
5. **Documentation Completion**: Test cases, defect reports, and release notes are finalized and approved with accessible and documented version information.